

CSSE 230 – Data Structures and Algorithms

[Typical Exam instructions]

You may use a calculator, a writing implement, and two sides of 8.5 x 11 note pages. You must turn in this written part before you use your computer, textbook, or any other resources.

You may not communicate with any person other than your instructor about this exam until allowed by your instructor. You may not use for any purpose email, chat, cell phones or any communication device until you have submitted your exam, unless you are given specific permission by your instructor. You may also not use any device with headphones or earbuds.

Part 2 (computer) rules

You may use any printed or written materials that you brought with you, code that you wrote yourself or with a partner before the exam, anything on the CSSE230 Moodle and web sites, and the Java API documentation (on your laptop or at oracle.com).

You may not otherwise search the internet or communicate with any person other than your instructor or a student assistant. You may not communicate with anyone else about this exam until allowed by your instructor.

You will sign a statement to certify that you neither received help from others nor gave it on either part of the exam.

You must actually get this problem working on your computer. Almost all of the credit for this problem will be for code that actually works.

At the end of the computer part, you will find this checklist.

Please check off each item below:

Turn-in checklist:

1. ☐ I signed the statement above.
2. ☐ I saved my work.
3. ☐ I committed my solution via SVN.
4. ☐ I saw in Eclipse that my solution was committed
5. ☐ I circled the number of the programming problem I want graded (if I was given a choice)
6. ☐ I am about to hand this test paper to my instructor or an assistant.

1. (10 points) When writing HeapSort, we discussed two different approaches to the sub problem of initially arranging elements 1-N of an array into a binary heap, and chose one of them because it was more efficient. Briefly describe the two approaches and explain why one is more efficient than the other.
2. (10 points) (Note: we recommend adding the statement of the master theorem to your notes pages.)

Write and solve (big-theta) a recurrence relation for the running time of the following method

```
public static void mystery(int n) {  
    if (n==1)  
        return;  
    mystery(n/10);  
    mystery(n/10);  
    for (int i=1; i<= n; i++)  
        nTimeMethod(i, n); // this function call always takes O(n) time.  
    mystery(n/10);  
}
```

3. (16 points) Someone writing a class to implement the **priority queue ADT** has many options of underlying data structures to use to contain the data. Briefly discuss the advantages and disadvantages of each proposed representation, in terms of the three **PriorityQueue** operations (insert, findMin, deleteMin). You should consider big-theta runtime of the main operations, programming ease, and memory requirements.

Structure	Advantage(s)	Disadvantage(s)
Unsorted array		
Sorted array		
Sorted linked list		
Binary Search Tree		
AVL tree		
Hash Table		
Binary Heap		

4. (15 points) In the EditorTrees project, you were given this definition of a node in a height-balanced binary tree with rank::

```
public class Node {  
    enum Code {SAME, LEFT, RIGHT};  
  
    char element;  
    Node left, right; // subtrees  
    Node parent;  
    int rank;          // inorder position of this node within its own subtree.  
    Code balance;
```

Having a balance code allowed us to determine which node in an AVL tree was the deepest node that became unbalanced (thus needed a rotation) as a result of inserting a single element. Please give pseudocode for an algorithm for doing this that operates in $O(\log n)$ time. (Note: pseudocode can use a mixture of Java and English, but is still precise).

Assumptions:

You have a reference to the node that was just inserted.

The balance codes have not yet been changed in the process of inserting the new node.

For this problem, you do not need to modify any balance codes or do any rotations.

Your pseudocode should output

NO ROTATION, STOPPED AT NODE WITH ELEMENT x	if no rotations are required, or
ROTATION AT NODE WITH ELEMENT x	if a rotation is required.

where x is the actual element of the node.

5. (12 points) Consider a very simple hashcode function for integers, $h(N) = N$. The following list of numbers is inserted (in the order given) into an initially empty table. Show where each number ends up in this table of size 12 if
- (a) linear probing, and (b) quadratic probing is used. Most of the points will be for correctly dealing with collisions.

16, 3, 29, 7, 15, 24, 11, 22, 4

a)

--	--	--	--	--	--	--	--	--	--	--	--

b)

--	--	--	--	--	--	--	--	--	--	--	--

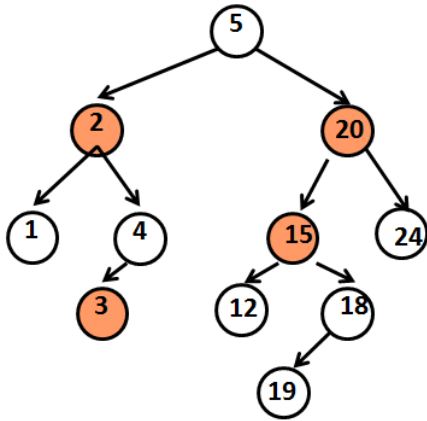
6. (10 pts) A binary tree whose elements are characters has the following properties:

(a) A level-order traversal of its ten nodes yields: **X J G R M P Y A W B.**

(b) An in-order traversal its ten nodes yields: **J Y R A X B W M G P.**

Draw the tree.

7. (6 pts) Is this a valid red-black BST or not? (The shaded nodes are red. The null external nodes aren't shown.) Hint: it violates three of the five red-black tree properties – which ones?



8. (10 pts) Write code that given a node *n*, gets the node's successor in an in-order traversal of the tree, or null if it is the last node in an in-order traversal. **Assume that each node has a pointer to its parent.** Note that this code is very different than most code we've written for trees, that is, you will probably find it easier to use a loop rather than recursion.

```
class Node {  
    int element;  
    Node left;  
    Node right;  
    Node parent;  
    ...  
}
```

```
public Node getInorderSuccessor(Node n) {
```

9. (5 points) (10 pts) Given two sets A and B represented as sorted lists (your choice of ArrayList or LinkedList), consider an algorithm for computing $A \text{ xor } B$ (the set of elements that are in A or B, but not both).
- a) Write code or pseudocode (Java syntax not required, as long as you are precise) to do this efficiently.
- b) What is the runtime of your algorithm? $\Theta(\text{_____})$

10. (5 pts) Suppose we call **QuickSort** on the array below. Assume 45 is chosen as the pivot, and that you start by swapping the pivot to position 0 so it matches the code we learned in class, and then swapping the pivot back to the right place when done. Show the array contents after a partition is done, but before the recursive calls.

11 18 81 9 55 10 77 98 45 50 78 22 38

11. (6 pts) List a common sort that has the same (exact) run-time on any array of a given size, regardless of the initial ordering of the elements.

_____ Name a second one: _____

12. (5 pts) What is the big-theta runtime (in terms of m , n , and b) of radix-sorting a set of n integers with values in the range $[0, m]$ when using b buckets?

13. (6 pts) Given an extended binary tree (EBT) of height h , **including** any links to external nodes in the calculation of the height,

...what is the *maximum* number of external nodes it may have? _____

...what is the *minimum* number of external nodes it may have? _____

14. (8 pts) Describe (in words and/or diagrams) the **set of all** trees whose preorder and postorder traversals visit the nodes in opposite order.

15. (4 pts) If a height-balanced tree has height h , give a recurrence for the *minimum* number of nodes it can have. Then solve that recurrence to find a closed-form solution. (Note: we did this on the day 12 quiz even before we understood the term *recurrence*.)